AESC / NeuroField v2: A Two-Speed Memory Architecture for Recurrent Language Models

Giridhari Karmakar
giriisdev@gmail.com

Technical research report · Project AERIS_64 · October 2026

Abstract

This report describes **NeuroField (AESC v2)**, a recurrent language-model architecture that combines a predictive dendrite, a k-step neural field, top-k skill routing, and a **two-speed memory**: fast weights written token-by-token by a surprise-gated delta rule, and slow weights consolidated by replay. The production code path uses a **parallel / looped form** (depth passes over the sequence with a causal fast-memory scan) rather than a pure Transformer attention stack. We document forward equations, safety freeze behaviour, training practice, empirical status on small runs, and known limitations. The architecture is operational in open-source code; scale and frontier-quality language performance remain open and are left for future work.

1. Introduction

Large language models typically rely on stacked self-attention. AESC explores a complementary hypothesis: local predictive structure (dendrite), iterative state refinement (neural field), modular skills (router), and explicit fast/slow memory can support sequence modelling without a full T×T attention matrix. The project name **AERIS_64** denotes the engineering product shell around this architecture. This document is an honest engineering–research snapshot: the design runs end-to-end (train, checkpoint, serve), but data scale and optimisation gaps still limit quality.

2. Architecture overview

At a high level, a token sequence is embedded, passed through a causal dendrite window, then refined over *k* depth passes. Each depth applies prediction and gating, skill routing, a causal delta-rule memory update, and a neural-field step. A slow ring provides longer context. Logits are read out from skills plus fast and slow memory projections.

```
tokens → Embed
       → Dendrite window (causal local past)
       → for depth j = 1 .. k:
            Predict → delayed error E
            Gate g (neuromodulator)
            Router / skills → O
            Fast memory causal scan (delta rule)
            Neural field step
       → Slow ring read (+ segment write)
       → LN(O + scale·P_f(R) + P_s(U)) → logits
```

3. Forward equations

3.1 Embedding

```
X_t = Embed(token_t) ∈ R^{d}
```

3.2 Dendrite (causal local window + delayed error)

With window width W and learned coefficients c_j, bias b, and error gain γ:

```
d^{win}_t = tanh( Σ_{j=0}^{W-1} c_j ■ x_{t-j} + b )
d_t       = d^{win}_t + γ ■ e_t
```

3.3 Prediction, error, and surprise

```
x■_{t+1} = f_p(h_t)
e_{t+1} = sg(x_{t+1}) − x■_{t+1}     # detached target (recommended)
S_{t+1} = sg( mean |e_{t+1}| )
S■_t    = ρ S■_{t−1} + (1−ρ) S_t
S■_t    = S_t / (S■_t + ε)
```

3.4 Write gate (neuromodulator)

```
g_t = σ( a_s (S■_t − 1) + a_c mean(|h_t|) + b + δ_t )
g_t ← min(g_t, g_max)

Safety freeze (paper §7 style): if memory_frozen then g_t = 0
  ⇒ M only decays, no new writes.
```

3.5 Neural field (one step; repeated k times in sequential form)

```
h ← (1−ζ) ■ h + ζ ■ LN( α ■ h + β ■ ( d_t + R(tanh h) + W_m r_{t−1} ) )

R = shared local conv over neuron index
k_depth may be fixed or scheduled from relative surprise.
```

3.6 Router and skills

```
π = softmax(W_r h)
w = softmax(top-k logits of π)
w^{STE} = w + π − sg(π)          # straight-through estimator
o_t = Σ_i w^{STE}_i Op_i(h)
```

3.7 Fast memory — decay-consistent delta rule (Change 5)

```
k_t = normalize(W_k d_{t−1})
v_t = W_v o_t
M^Λ = Λ M_{t−1}                  # Λ = diag(σ(θ)), elementwise decay
M_t = M^Λ + g_t k_t (v_t − (M^Λ)^T k_t )^T
M_t ← M_t · min(1, M_max / ||M_t||_F)   # optional bound

Read:
q_t = normalize(W_q d_t)
r_t = M_t^T q_t

If g_t = 0: M_t = Λ M_{t−1} (Proposition: decay only).
```

3.8 Slow memory and readout

```
u_t = SlowRead(d_t; ring slots)     # content-addressed ring
z_t = W_o LN( o_t + s · P_f r_t + P_s u_t )
logits_t = z_t over vocabulary

Replay sketch:
W_s ← (1−η) W_s + η Σ_b (S^{(b)} / Σ S) M^{(b)}
```

3.9 Parallel / looped production path

The shipped forward pass embeds all T positions, builds dendrite windows in parallel, then runs k depth iterations. Within each depth, projections and routing are batched over T; the fast-memory delta rule remains a **causal left-to-right scan**. Each depth uses a fresh fast-memory matrix to avoid cross-depth leakage of future writes into early positions. Decode-time token-by-token generation must match this math (parity tests).

```
X = Embed(tokens)                 # (B, T, d)
D_win = DendriteWindow(X)         # causal
H = D_win
for j in 1..k:
    E = sg(X) − shift(Predict(H))
    D = D_win + γ ■ E
    O = RouterSkills(H)           # all T
    M, R = DeltaScan_causal(D, O, G)
    H = FieldStep(H, D, shift(R))
logits = Readout(O, R, U_slow)
```

4. Causality

Unlike Transformers, AESC does not use a triangular attention mask. Causality is enforced by (i) the dendrite window over past embeddings only, (ii) one-step-shifted prediction error, (iii) ordered fast-memory writes, and (iv) shifted memory reads into the field. Changing x_{t+1} must not change logits at positions ≤ t (leakage tests).

5. Training

Training minimises next-token cross-entropy, plus optional error-energy and routing balance terms. Practical additions from standard LM practice include label smoothing, dropout, warmup, and cosine or Noam-style learning-rate schedules. Data-parallel training via torchrun/DDP is supported. Throughput is typically lower than a pure Transformer of similar width because of the causal memory scan and depth loop; GPU batching, shorter sequences, reduced k, and multi-GPU are the primary speed levers.

```
L = CE(logits, targets; label_smoothing)
  + λ_err · mean(||E||^2)
  + λ_bal · routing_balance
```

6. Safety observability

Section-7 style design: anomaly detection can set a session flag memory_frozen, forcing g=0 so that fast memory only decays. This avoids destructive permanent weight freezes while stopping further plastic writes under suspicious surprise patterns.

7. Implementation status

Open-source repository AERIS_64 provides: core modules (dendrite, field, memory, router, neuromodulator), parallel forward, checkpointing (safetensors), training entrypoints, FastAPI serve, optional tools, and tests (including leakage / decode parity where maintained). Small educational runs confirm loss decrease and causal checks; they are not claims of competitive language modelling.

8. Limitations and future work

The author acknowledges present gaps, to be addressed when time permits:

• **Data scale and SFT quality** dominate perceived “intelligence”; architecture alone is insufficient.
• **Throughput**: causal delta scan + depth k makes large token budgets expensive (days vs hours).
• **Gate collapse** ($g \approx 0$) can disable plasticity; needs monitoring and schedule tuning.
• **Train vs decode parity** remains a recurring engineering risk under path changes.
• **Skill utilisation** and long-context ring behaviour need multi-seed, harder tasks.
• **Chunkwise / fully parallel delta algorithms** and stronger baselines vs Transformers are open.
• Safety policy beyond freeze is thin; red-teaming incomplete.

9. Conclusion

NeuroField / AESC v2 is a coherent two-speed-memory recurrent design with explicit equations, a working codebase, and clear causality mechanisms. The architecture is suitable for continued research and small-scale deployment experiments. Closing the gap to strong language performance requires data, compute, and the limitation list above—not a redesign of the core idea.

Author

Giridhari Karmakar
Email: giriisdev@gmail.com
Project: AERIS_64 / NeuroField (AESC)
Repository: https://github.com/giridhari-code/AERIS_64

This document is a technical research report for archival and engineering handoff. It does not claim peer-reviewed publication status.